Assignment 3:
Mohammed Imtiyaz
FAU ID Z23749265

```
#1  answer
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat

from google.colab import drive
drive.mount('/content/drive')
data = loadmat('/content/drive/MyDrive/allFaces.mat')

# Load the .mat file
#data = loadmat('/content/allFaces.mat')

# Extract the 'faces' and 'nfaces' variables from the loaded data
faces = data['faces']
nfaces = data['nfaces'][0]  # Assuming 'nfaces' is a 2D array and we
need the first row

# Image dimensions
n = 192  # Height of the image
m = 168  # Width of the image

# Function to plot the first image of the first 36 individuals
def plot_faces(faces, nfaces, n, m):
    # Initialize a container to store the first picture of the first 36
persons
    Person = np.zeros((n, m, 36))

    # Plotting the first pictures of person #1 to person #36
    plt.figure(figsize=(15, 15))
    for i in range(36):
        # Calculate the starting index for each person
        start_index = sum(nfaces[:i])
        # Reshape the column vector to the original image size,
ensuring the correct order
        Person[:, :, i] = np.reshape(faces[:, start_index], (n, m),
order='F')
        plt.subplot(6, 6, i+1)
        plt.imshow(Person[:, :, i], cmap='gray')
        plt.axis('off')
        plt.gca().set_aspect('equal', adjustable='box')

    plt.show()

plot_faces(faces, nfaces, n, m)
```



```
import scipy.io
import numpy as np
import matplotlib.pyplot as plt

from scipy.io import loadmat

from google.colab import drive
drive.mount('/content/drive')
data = loadmat('/content/drive/MyDrive/allFaces.mat')

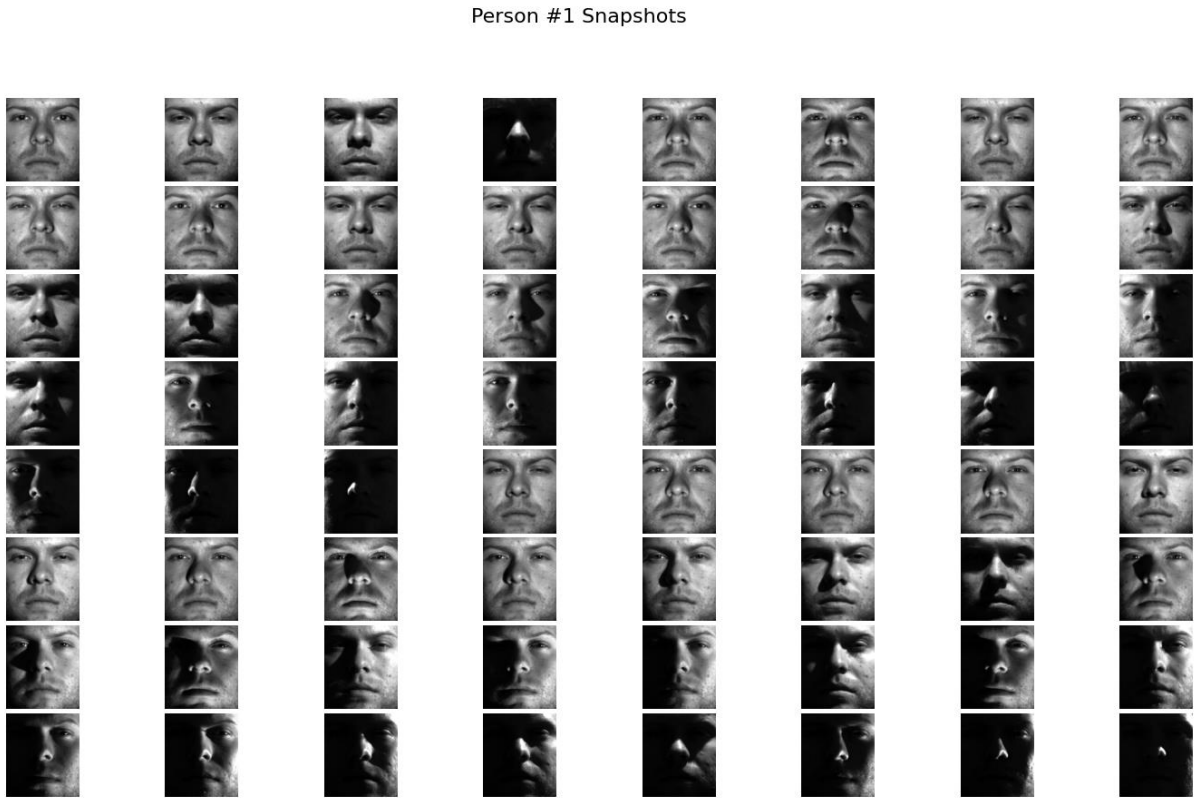
# Extracting the faces data and the nfaces vector, which contains the
number of pictures for each person
faces = data['faces']
```

```
nfaces = data['nfaces'][0]
n = 192  # height of the image
m = 168  # width of the image

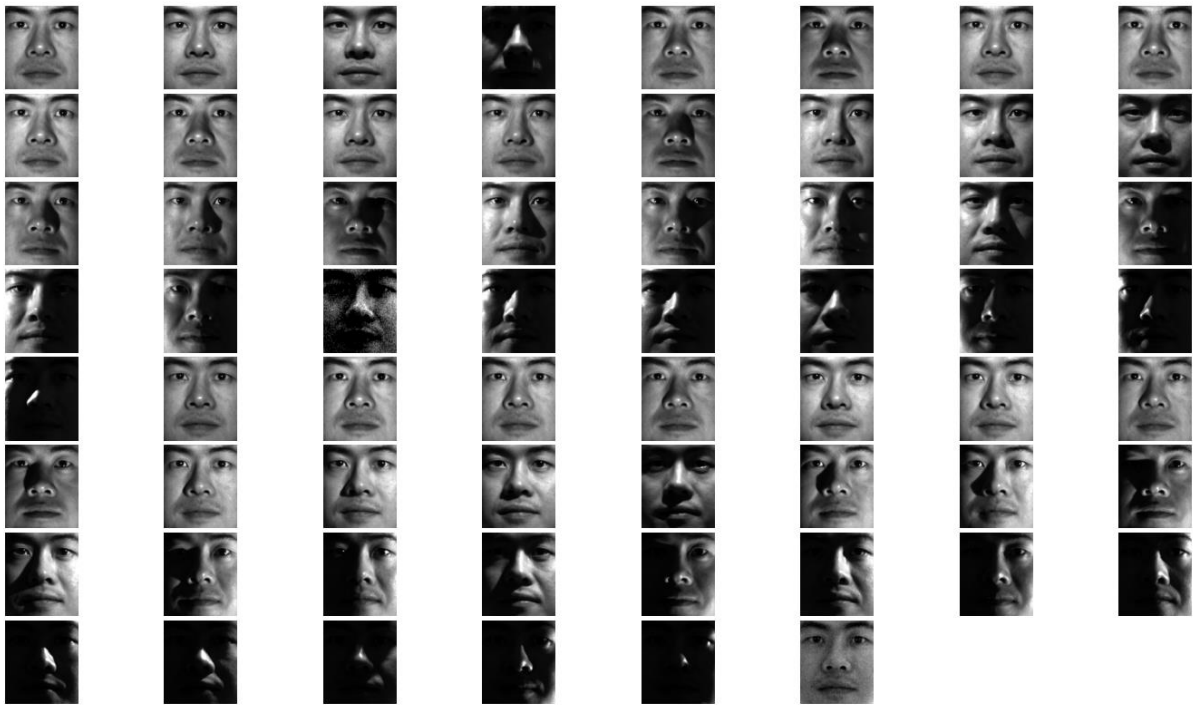
# Displaying images for the first two people for demonstration
for i in range(2):
    start_col = sum(nfaces[:i]) if i > 0 else 0
    end_col = sum(nfaces[:i+1])
    subset = faces[:, start_col:end_col]

    # Number of plots to display is limited by either 64 or the actual
number of pictures available for the person
    plots_to_display = min(nfaces[i], 64)


    plt.figure(figsize=(18, 10))
    for j in range(plots_to_display):
        Snapshot = np.reshape(subset[:, j], (m, n)).T
        plt.subplot(8, 8, j + 1)
        plt.imshow(Snapshot, cmap='gray')
        plt.axis('off')
    plt.subplots_adjust(wspace=0.05, hspace=0.05)
    plt.suptitle(f'Person #{i+1} Snapshots', fontsize=16)
    plt.show()
```



Person #2 Snapshots



2

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat


from google.colab import drive
drive.mount('/content/drive')
data = loadmat('/content/drive/MyDrive/allFaces.mat')

# Extract the 'faces' array from the loaded data
faces = data['faces']

# Calculate the average face
average_face = np.mean(faces, axis=1)

# Subtract the average face from all face images
faces_centered = faces - average_face[:, None]

# Store these average subtracted faces in matrix X
X = faces_centered

# Assuming the image dimensions are 192x168 (from previous messages)
image_height = 192
image_width = 168
```

```
# Function to reshape and display an image correctly
def display_image(image_vector):
    # Reshape the image vector to 2D dimensions (image_width,
image_height)
    image_reshaped = image_vector.reshape(image_width, image_height).T
    plt.imshow(image_reshaped, cmap='gray')
    plt.axis('off')  # Hide the axis
    plt.show()

# Display the average face
print("Average Face:")
display_image(average_face)
```



3,

```
# Calculating the SVD of X using a more memory-efficient method,
focusing on a subset if needed

U, S, Vt = np.linalg.svd(X[:, :2410], full_matrices=False)
# Now, we reshape the first 54 U vectors to 2D pictures to get the
eigenfaces
eigenfaces = U[:, :54].reshape(n, m, 54)

fig, axes = plt.subplots(6, 9, figsize=(18, 12))
for i, ax in enumerate(axes.flatten()):
    eigenface_corrected = np.rot90(eigenfaces[:, :, i], k=3)  # Example
with k=3 for counterclockwise rotation
    eigenface_flipped = np.fliplr(eigenface_corrected)
```

```
    ax.imshow(eigenface_flipped, cmap='gray')
    ax.axis('off')
plt.tight_layout()
plt.show()
```



Yes, I think each of the eigenface appears to capture different features or variations in facial features. Some represent the variation in lighting across the face, others represent basic components of facial features such as eyes, nose, mouth, or the outline of the head. Some eigenfaces might capture more abstract features not directly related to specific facial parts but rather to variations between faces.

```
# Extract the first and fifth eigenfaces as vectors (before reshaping)
first_eigenface_vector = U[:, 0]
fifth_eigenface_vector = U[:, 4]

# Compute the inner product of the first and fifth eigenfaces
inner_product_1_5 = np.dot(first_eigenface_vector,
fifth_eigenface_vector)

# Extract the 10th and 15th eigenfaces as vectors
tenth_eigenface_vector = U[:, 9]
fifteenth_eigenface_vector = U[:, 14]

# Compute the inner product of the 10th and 15th eigenfaces
```

```
inner_product_10_15 = np.dot(tenth_eigenface_vector,
fifteenth_eigenface_vector)

inner_product_1_5, inner_product_10_15
```

output:

(3.0531133177191805e-16, 1.942890293094024e-16)

These values are extremely close to zero , can be considered as effectively zero.

we can say that the eigenfaces are orthogonal.

4,

```
# The first picture of person #37 is located immediately after the last
picture of person #36 in the dataset.
# We need to calculate the index for the first picture of person #37
based on the nfaces vector.
index_person_37_first_picture = sum(nfaces[:36])

# Extract the first picture of person #37 from the matrix X
v_person_37_first_picture = X[:, index_person_37_first_picture]

# The maximum value of r is the rank of the matrix X or the number of
eigenfaces, whichever is smaller.
max_r = min(U.shape[1], v_person_37_first_picture.size)

# Compute the coefficients (alphas) for the approximation using the
first r eigenfaces, with r = 5 here.
r = 5
alphas = [np.dot(U[:, i].T, v_person_37_first_picture) for i in
range(r)]

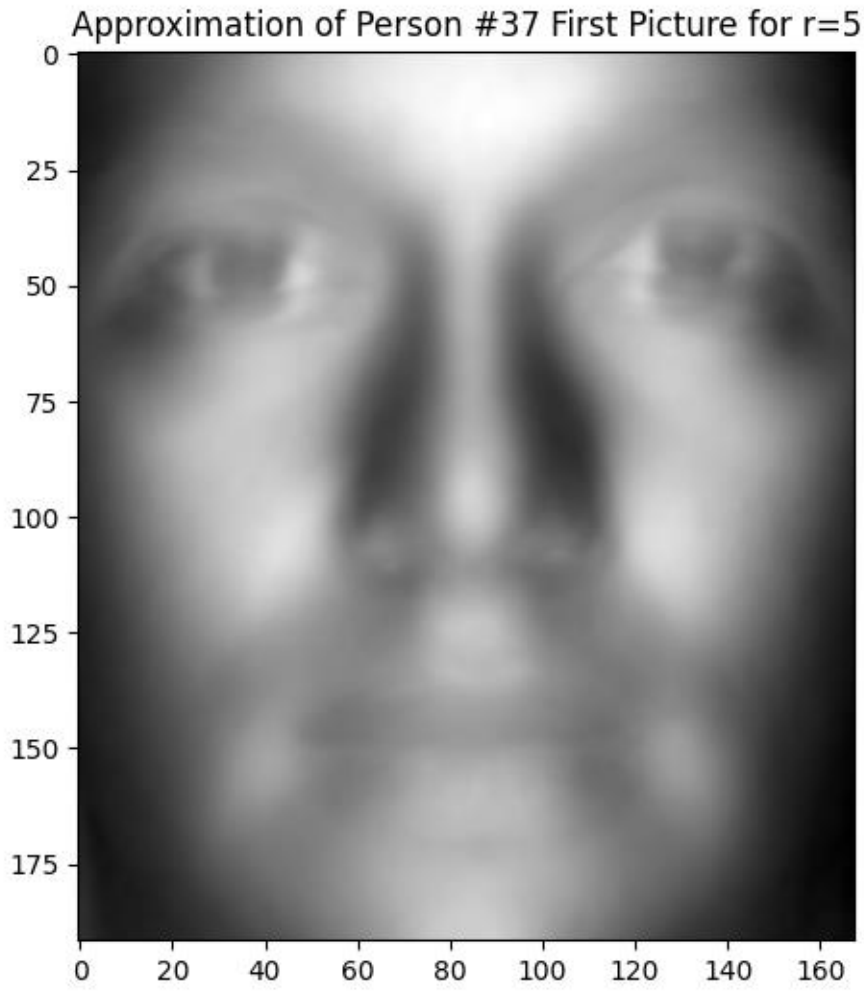
# Recreate the approximation of V using the first r eigenfaces
v_approx_r5 = np.sum(np.array([alphas[i] * U[:, i] for i in range(r)]),
axis=0)

v_approx_r5_2d = np.rot90(v_approx_r5.reshape(n, m),k=3)

# Reshape the approximation for visualization as a 2D picture
#v_approx_r5_2d = v_approx_r5.reshape(n, m)

# Plot the approximation of vector V for r=5
plt.figure(figsize=(6, 6))
plt.imshow(v_approx_r5_2d, cmap='gray')
plt.title('Approximation of Person #37 First Picture for r=5')
plt.show()

max_r
```

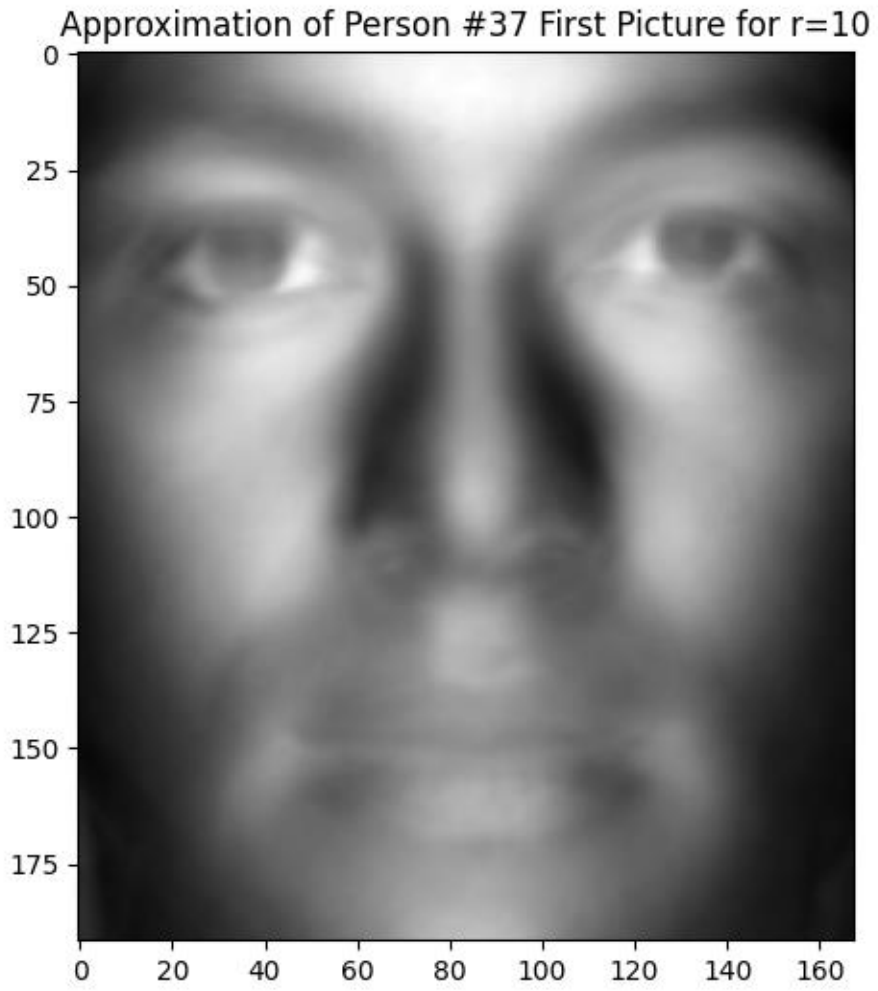



```
# For r = 10
r = 10
alphas_r10 = [np.dot(U[:, i].T, v_person_37_first_picture) for i in
range(r)]

# Recreate the approximation of V using the first r eigenfaces
v_approx_r10 = np.sum(np.array([alphas_r10[i] * U[:, i] for i in
range(r)]), axis=0)

# Reshape the approximation for visualization as a 2D picture
v_approx_r10_2d = np.rot90(v_approx_r10.reshape(n, m),k=3)

# Plot the approximation of vector V for r=10
plt.figure(figsize=(6, 6))
plt.imshow(v_approx_r10_2d, cmap='gray')
plt.title('Approximation of Person #37 First Picture for r=10')
plt.show()
```

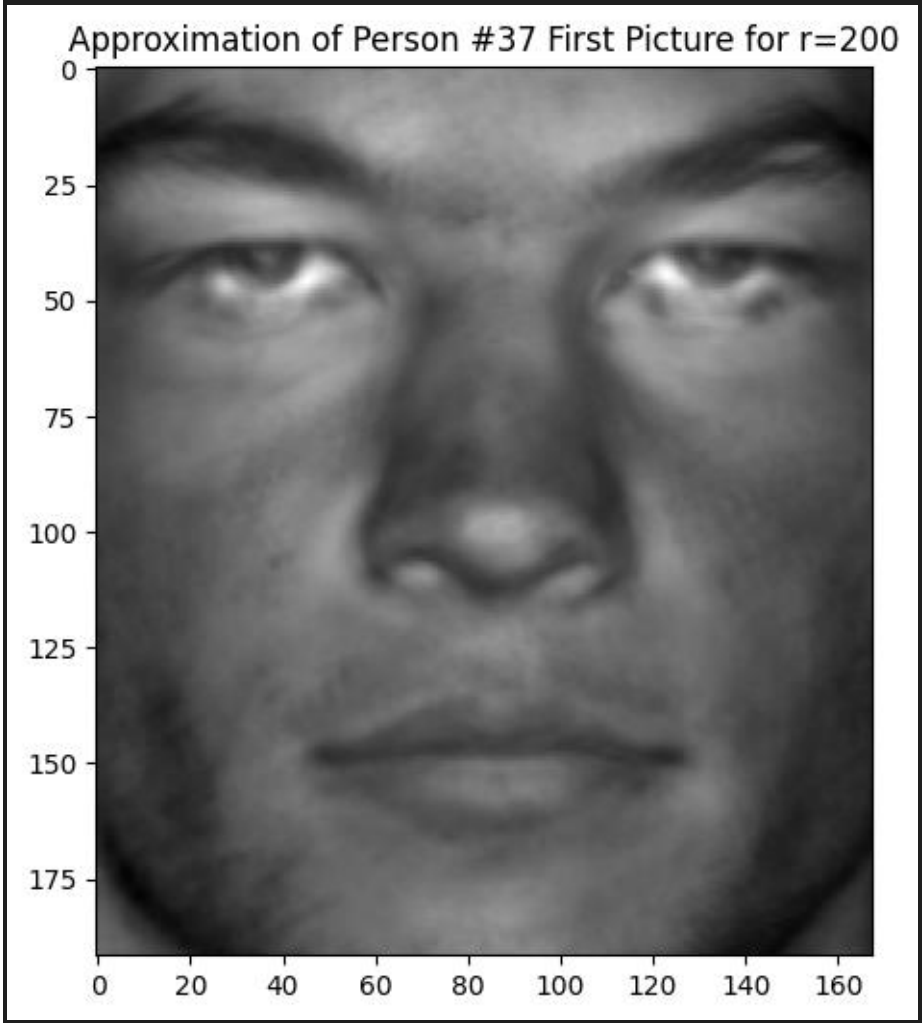



```
# For r = 200
r = 200
alphas_r200 = [np.dot(U[:, i].T, v_person_37_first_picture) for i in
range(r)]

# Recreate the approximation of V using the first r eigenfaces
v_approx_r200 = np.sum(np.array([alphas_r200[i] * U[:, i] for i in
range(r)]), axis=0)

# Reshape the approximation for visualization as a 2D picture
v_approx_r200_2d = np.rot90(v_approx_r200.reshape(n, m),k=3)

# Plot the approximation of vector V for r=200
plt.figure(figsize=(6, 6))
plt.imshow(v_approx_r200_2d, cmap='gray')
plt.title('Approximation of Person #37 First Picture for r=200')
plt.show()
```

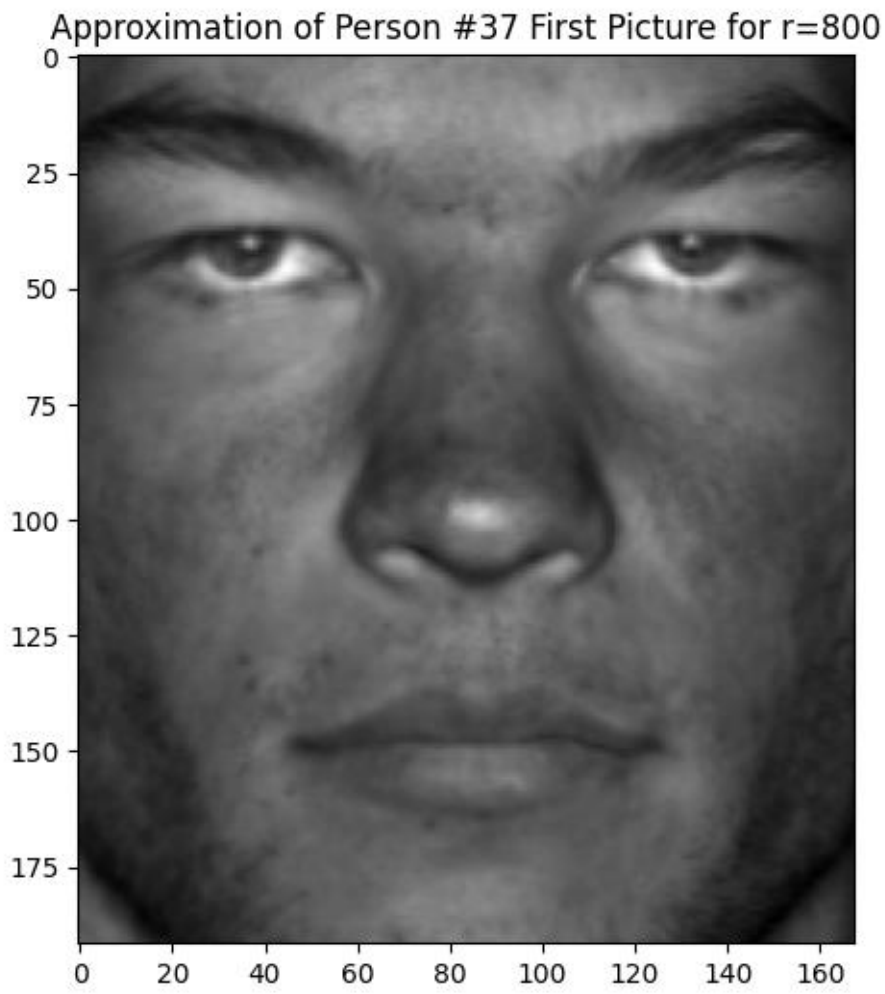



```
# For r = 800
r = 800
alphas_r800 = [np.dot(U[:, i].T, v_person_37_first_picture) for i in
range(r)]

# Recreate the approximation of V using the first r eigenfaces
v_approx_r800 = np.sum(np.array([alphas_r800[i] * U[:, i] for i in
range(r)]), axis=0)

# Reshape the approximation for visualization as a 2D picture
v_approx_r800_2d = np.rot90(v_approx_r800.reshape(n, m),k=3)

# Plot the approximation of vector V for r=200
plt.figure(figsize=(6, 6))
plt.imshow(v_approx_r800_2d, cmap='gray')
plt.title('Approximation of Person #37 First Picture for r=800')
plt.show()
```




```
# For r = 100, ensuring not to exceed the maximum number of available
eigenfaces
r = min(100, max_r)  # Ensuring r does not exceed the maximum number of
eigenfaces
alphas_r100 = [np.dot(U[:, i].T, v_person_37_first_picture) for i in
range(r)]

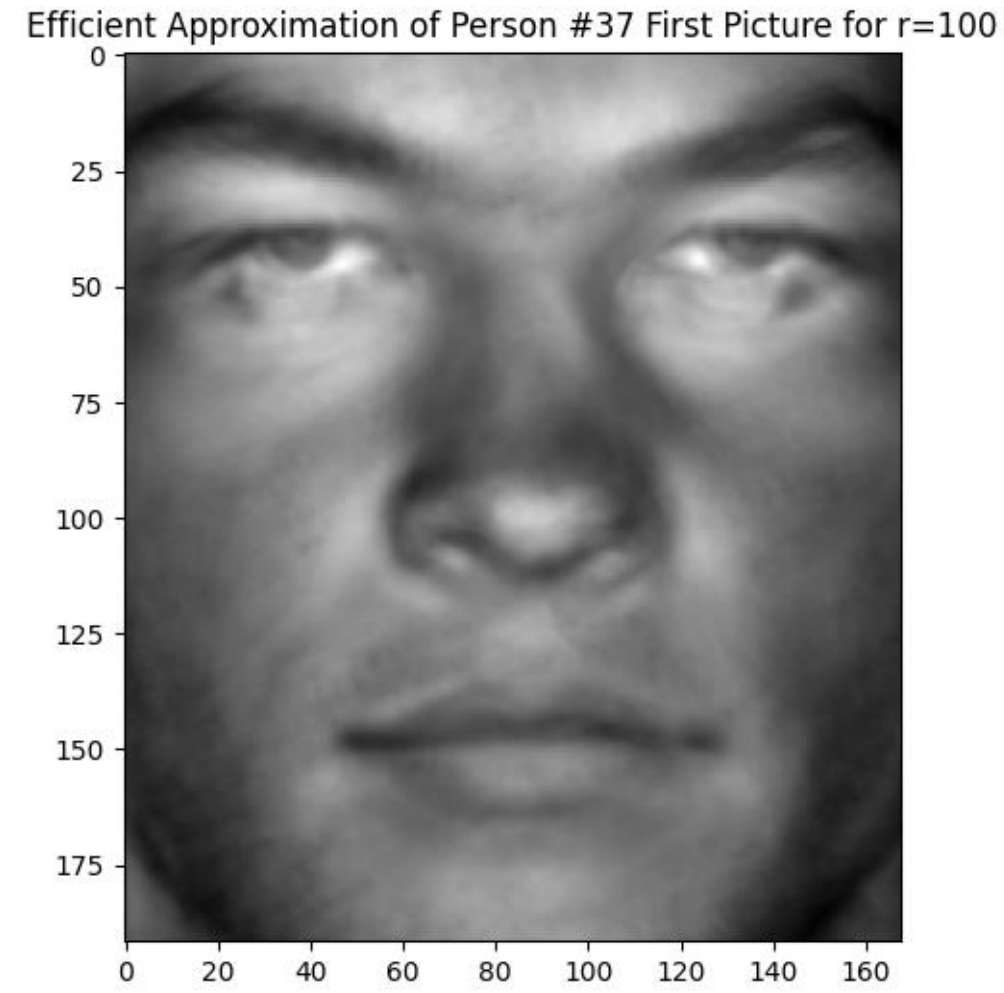
# Initialize an array for the approximation
v_approx_r100_efficient = np.zeros(U.shape[0])

# Accumulate the contributions directly
for i in range(r):
    v_approx_r100_efficient += alphas_r100[i] * U[:, i]

# Reshape for visualization
```

```
v_approx_r100_2d_efficient =
np.rot90(v_approx_r100_efficient.reshape(n, m),k=3)

# Plot the approximation of vector V for r=100 using the more memory-
efficient approach
plt.figure(figsize=(6, 6))
plt.imshow(v_approx_r100_2d_efficient, cmap='gray')
plt.title('Efficient Approximation of Person #37 First Picture for
r=100')
plt.show()
```



Yes, there is definitely good approximation of the picture with r=100, but not sure if this will be true for all images. In this case r=100 seems to be good.

5,

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.io import loadmat

from google.colab import drive
```

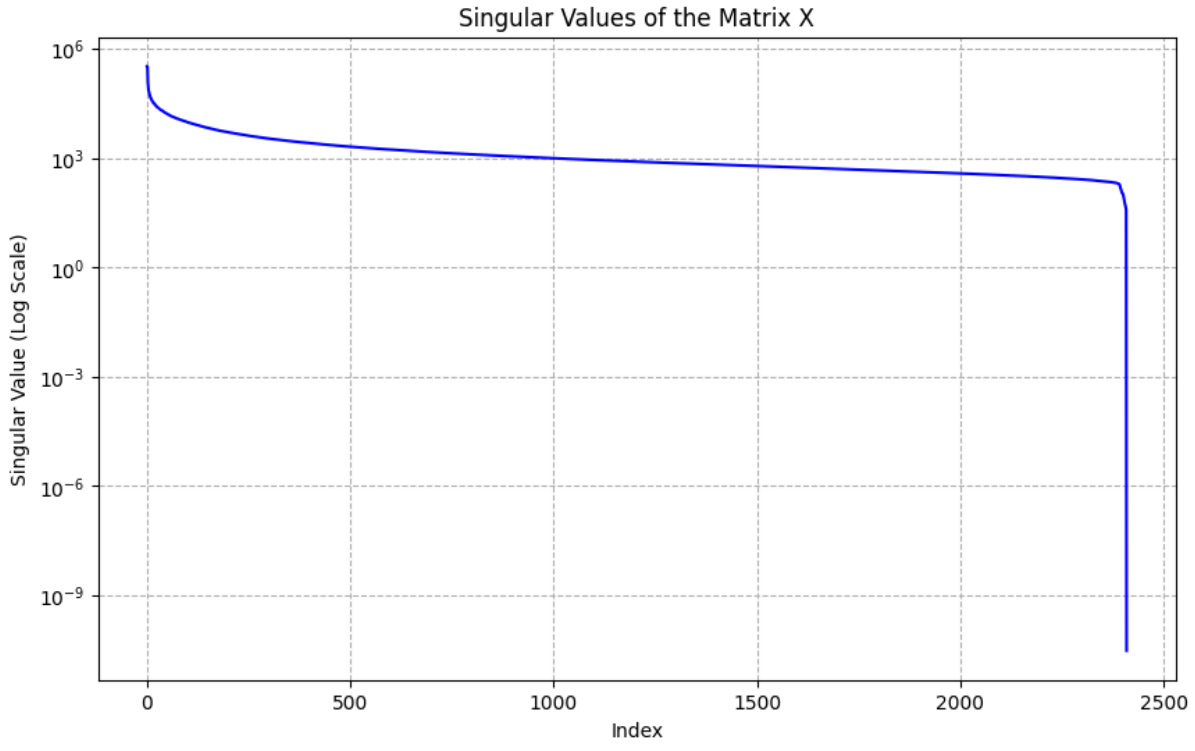
```
drive.mount('/content/drive')
data = loadmat('/content/drive/MyDrive/allFaces.mat')

# Extracting the faces matrix from the loaded data
faces = data['faces']

# Computing the average face and subtract it from the faces matrix to
center the data
average_face = np.mean(faces, axis=1)
X = faces - average_face[:, None]

# Performing Singular Value Decomposition (SVD) on the centered faces
matrix
U, S, Vt = np.linalg.svd(X, full_matrices=False)

# Plotting the singular values on a semi-logarithmic scale
plt.figure(figsize=(10, 6))
plt.semilogy(S,  linestyle='-', color='b')
plt.title('Singular Values of the Matrix X')
plt.xlabel('Index')
plt.ylabel('Singular Value (Log Scale)')
plt.grid(True, which="both", ls="--")
plt.show()
```



A good truncation point would be right before the curve flattens, which seems to be around the index where the curve starts to become horizontal.Good point for truncation could be 800 to 1500.

```
6, import numpy as np
```

```
import matplotlib.pyplot as plt
from scipy.io import loadmat

from google.colab import drive
drive.mount('/content/drive')
data = loadmat('/content/drive/MyDrive/allFaces.mat')
# Extract faces, nfaces, and dimensions from the loaded data
faces = data['faces']
nfaces = data['nfaces'][0]
m, n = 192, 168  # Assuming the image dimensions are 192x168 as before

# Calculate the average face and subtract it from the faces to center
the data
average_face = np.mean(faces, axis=1)
X = faces - average_face[:, None]

# Perform SVD to get the eigenfaces
U, S, Vt = np.linalg.svd(X, full_matrices=False)

# Get the indices for the images of persons 2 and 7
person_2_indices = slice(sum(nfaces[:1]), sum(nfaces[:2]))  # Person 2
person_7_indices = slice(sum(nfaces[:6]), sum(nfaces[:7]))  # Person 7

# Project the images of person 2 and person 7 onto the 5th and 6th
eigenfaces
projection_5th = U[:, 4]
projection_6th = U[:, 5]

# Calculate the coefficients for the projection onto the 5th and 6th
eigenfaces
coeffs_person_2_5th = np.dot(projection_5th.T, X[:, person_2_indices])
coeffs_person_2_6th = np.dot(projection_6th.T, X[:, person_2_indices])
coeffs_person_7_5th = np.dot(projection_5th.T, X[:, person_7_indices])
coeffs_person_7_6th = np.dot(projection_6th.T, X[:, person_7_indices])

# Plot the projections onto the 5th and 6th eigenfaces for persons 2
and 7
plt.figure(figsize=(8, 6))
plt.scatter(coeffs_person_7_5th, coeffs_person_7_6th, c='red',
label='Person 7')
plt.scatter(coeffs_person_2_5th, coeffs_person_2_6th, c='black',
label='Person 2')
plt.xlabel('5th Eigenface Coefficient')
plt.ylabel('6th Eigenface Coefficient')
plt.title('Projection onto 5th and 6th Eigenfaces')
plt.gca().invert_yaxis()
plt.gca().invert_xaxis()
plt.legend()
```

```
plt.grid(True)
plt.show()
```

